

AI Travel Planner — Technical Plan & 12-Day Build Schedule

Portfolio project. Goal: a finished, deployed, demo-able app in 10–14 days that proves you can do AI integration properly — not a toy that prints LLM text.

0. The honest framing: what actually makes this CV-worthy

Anyone can call an LLM and render the answer. That is a weekend tutorial and recruiters recognise it instantly. The four things that separate this from a tutorial, and that you should be able to talk about in an interview:

- 1. **Grounded generation.** The LLM never invents restaurants or museums. You retrieve *real* places from a Places API first, then pass that list to the model and let it *arrange* them into an itinerary. This is retrieval-then-generate. It kills hallucination, and it is the single most important architectural decision here.
- 2. **Structured output, validated.** The model returns JSON matching a schema you define. You validate with Zod before it touches the DB. You handle the case where it returns malformed JSON (it will, eventually).
- 3. **Cost and quota control.** Cache place lookups and generated itineraries in Postgres. Rate-limit per user. Be able to say "a repeat destination costs me zero API calls."
- 4. **Streaming UX.** Itinerary appears progressively instead of a 20-second spinner.

If you build only those four things well, the project is strong even with a plain UI.

1. API & data stack — verified September 2026

1.1 The LLM

Option	Reality check
Google Gemini (AI Studio) ★ recommended	Permanent free tier, no credit card, Flash / Flash-Lite tiers. Note: free-tier prompts may be used to train Google's models, and enabling billing <i>replaces</i> the free allowance

Option	Reality check
	rather than adding to it. Google stopped publishing fixed quota numbers — check the live quota in AI Studio for your project. Pro-tier models left the free tier around April 2026.
OpenAI	No usable free tier in practice; a ~\$5 deposit is the effective minimum. Fine if you'd rather have OpenAI on the CV.
Anthropic (Claude)	No permanent free API tier. Pay per token.
Groq / Mistral	Free and fast, weaker at long structured JSON. Good fallback provider.

Decision: start on Gemini Flash (free), but put it behind an `LLMProvider` interface so swapping to OpenAI is a one-file change. Being able to say "provider-agnostic" in an interview is worth the 30 minutes it costs.

- Sign up: <https://aistudio.google.com>
- Use the **Vercel AI SDK** (`ai + @ai-sdk/google`) — it gives you streaming and `generateObject()` with Zod schema enforcement for free, across providers.

1.2 Maps & places — read this before you touch Google

Google Maps Platform changed materially in March 2025 and most tutorials are now wrong:

- The universal **\$200/month credit is gone**. Replaced by per-SKU monthly free caps: roughly **10,000 calls for Essentials, 5,000 for Pro, 1,000 for Enterprise** SKUs.
- A **billing account with a real payment method is required** to create a key, even if you never exceed the free tier. New accounts get a one-time \$300 trial credit.
- **There is no hard billing cap**. You can set budget alerts and per-API quotas, but alerts notify you *after* the spend. Set **per-API daily quotas in Cloud Console** — that is your actual safety net.
- Place Details / Nearby Search on the Pro & Enterprise field tiers are the expensive SKUs (reported around \$32/1k for advanced Place Details). Use **field masks** on Places API (New) to request only the fields you need — field mask determines which SKU you're billed at.

Two viable stacks:

Stack A — Google (best data, needs a card):

- Places API (New) [searchText](#) / [searchNearby](#) for POIs
- Geocoding API for destination → coordinates
- Maps JavaScript API for rendering
- Restrict the key by HTTP referrer + by API, set daily quotas, cache everything.

Stack B — all-free, no credit card ★ recommended for a student portfolio:

- **Map rendering:** MapLibre GL JS (open source) + free tiles, or Leaflet. No key.
- **Geocoding:** Nominatim (free, no key, **1 req/sec**, custom User-Agent required) or **Geoapify** (~3,000 requests/day free).
- **POIs:** Geoapify Places (OpenStreetMap-backed, category search, GeoJSON responses — no reviews or ratings) or **Foursquare Places** (richer venue data, small free tier).
- **Descriptions & photos:** Wikipedia / Wikidata REST API — free, no key, generous. Great for "about this place" copy.
- **Weather:** Open-Meteo — free, no API key, ideal for a packing-tips feature.
- **Images:** Unsplash or Pexels API, free with attribution.

Decision: build Stack B behind a [PlacesProvider](#) interface. If you later want Google data for the demo, you implement one adapter. The interface is the portfolio value.

1.3 Optional flights/hotels (only if you're ahead of schedule)

- **Amadeus Self-Service** — free monthly quota, test environment ([test.api.amadeus.com](#), 10 TPS, cached/limited data). Production keeps the same free quota and only charges beyond it (~fractions of a cent per call). Key + secret, OAuth2 token flow.
- Honestly: **cut this**. It's a whole extra auth flow and it isn't what makes the project impressive. Put it in the README as "future work."

1.4 Infrastructure

Concern	Pick	Why
Framework	Next.js (App Router) + TypeScript	Server Actions / Route Handlers keep API keys server-side
DB	Neon or Supabase Postgres (free tier)	serverless Postgres, works with Vercel

Concern	Pick	Why
ORM	Drizzle (or Prisma if you already know it)	Drizzle is lighter, TS-native, fast migrations
Auth	Auth.js (NextAuth) with Google OAuth	one provider only, don't build email/password
UI	Tailwind + shadcn/ui	fast, looks professional, no design skill needed
Validation	Zod	shared between LLM output, forms, and API routes
Deploy	Vercel	zero-config for Next.js
Rate limit	Upstash Redis free tier (or a simple Postgres table)	

1.5 `.env.local` you will end up with

LLM

GOOGLE_GENERATIVE_AI_API_KEY=

Places (Stack B)

GEOAPIFY_API_KEY=

FOURSQUARE_API_KEY= # optional

or Stack A

GOOGLE_MAPS_API_KEY= # server-side, unrestricted by referrer

NEXT_PUBLIC_MAPS_BROWSER_KEY= # referrer-restricted, rendering only

Images (optional)

UNSPLASH_ACCESS_KEY=

Database

DATABASE_URL=

Auth

AUTH_SECRET=

AUTH_GOOGLE_ID=

AUTH_GOOGLE_SECRET=

NEXTAUTH_URL=http://localhost:3000

Never expose an LLM or Places key with **NEXT_PUBLIC_**. All generation happens in a Route Handler or Server Action.

2. Architecture

Browser (React)

| POST /api/trips/generate { destination, days, budget, interests[] }



Route Handler (server)

1. authenticate (Auth.js session)
2. rate-limit check
3. cache check → if identical request exists, return cached trip
4. geocode destination → lat/lng
5. fetch candidate POIs by category (sights / food / nightlife / nature)
→ 40–60 real places, cached in `places` table
6. build prompt: user constraints + the candidate list (id, name, category, coords, price level)
7. LLM call with Zod schema → { days: [{ dayNumber, title, items:[{placeId,

```
startTime, durationMin, note, estimatedCost}} ], budgetBreakdown, tips }
```

8. validate; if invalid → one repair retry → else fail gracefully
9. resolve placeIds back to full place records (never trust LLM-invented data)
10. persist trip + stream to client

The rule that saves you: the LLM may only reference `placeId` values you gave it. Anything it invents gets dropped at step 9.

3. Data model (Drizzle/Postgres)

```
users      id, email, name, image, createdAt    -- from Auth.js
```

accounts/sessions -- Auth.js tables

destinations	id, slug, name, country, lat, lng, timezone, fetchedAt
--------------	--

places id, providerId, provider, destinationId, name, category,

lat, lng, address, priceLevel, rating, photoUrl,

description, raw jsonb, fetchedAt -- API CACHE

trips id, userId, destinationId, title, startDate, endDate,

numDays, budgetTotal, currency, interests text[],

promptHash, status, createdAt -- promptHash = cache key

trip_days	id, tripId, dayNumber, title, summary
-----------	---------------------------------------

trip_items id, tripDayId, placeId, position, startTime, durationMin,

note, estimatedCost

promptHash = SHA-256 of the normalised request. Same request → serve from DB, zero API cost. Mention this number in your README ("~85% cache hit rate in testing").

4. Twelve-day schedule

Each day has a **Definition of Done**. If a day isn't done, cut scope — do not roll it over.

Day 0 — environment (2h)

- `npx create-next-app@latest` (TS, App Router, Tailwind, ESLint, src dir)
- git init, GitHub repo, push
- Install Claude Code, add `CLAUDE.md` (see companion file)
- **DoD**: empty app running on `localhost:3000`, first commit pushed.

Day 1 — accounts, keys, spikes

- Create: Google AI Studio key, Geoapify key, Neon database.
- Write three throwaway scripts in `/scripts`: one LLM call, one geocode, one POI search. Print raw JSON. **Do not build UI today**.
- **DoD**: all three scripts return real data from your terminal. If an API fights you, today is the day to swap providers — not day 8.

Day 2 — schema & DB

- Drizzle schema for all tables above, `drizzle-kit push`, seed one destination manually.
- **DoD**: you can insert and query a trip from a script.

Day 3 — places layer

- `PlacesProvider` interface + Geoapify adapter.
- `getPlacesForDestination(destinationId, categories)` with DB cache-first logic.
- **DoD**: second call for the same city makes **zero** network requests. Prove it with a log.

Day 4–5 — the generation engine (the core; give it two full days)

- Zod schema for the itinerary.
- Prompt builder: system prompt + user constraints + candidate place list.
- `generateObject()` via Vercel AI SDK, temperature low, one repair retry on parse failure.
- Post-validation: drop unknown placeIds, clamp day count, check budget sum.
- **DoD**: `POST /api/trips/generate` returns a valid, fully-grounded itinerary JSON.

Day 6 — auth

- Auth.js + Google provider, Drizzle adapter, protected routes, session in layout.
- **DoD**: sign in, see your email, sign out. Trips are linked to `userId`.

Day 7 — the planning form + streaming result

- Form: destination autocomplete, number of days, budget, interest chips.
- Stream the generation with a skeleton itinerary.
- **DoD**: end-to-end from form submit to rendered itinerary in the browser.

Day 8 — itinerary UI

- Day tabs/timeline, place cards with photo, category, cost, note.
- Map panel (MapLibre) with numbered markers for the selected day.
- **DoD**: it looks like a product, not a JSON dump.

Day 9 — save & manage trips

- "My Trips" page, trip detail page, delete, public share link (`/t/[shareId]`).
- **DoD**: a logged-out person can open a shared trip link. This is your demo link.

Day 10 — hardening

- Rate limit (e.g. 10 generations/user/day). Error boundaries. Loading and empty states.
- Handle: LLM timeout, invalid JSON, zero POIs found, unknown destination.
- **DoD**: you cannot break it with bad input in 10 minutes of trying.

Day 11 — polish & deploy

- Deploy to Vercel, env vars, run migrations, seed 3–5 demo destinations so the demo is instant and cache-hot.
- Mobile responsive pass. Favicon, OG image, title.
- **DoD**: public URL that works on a phone.

Day 12 — the part everyone skips (do not skip it)

- **README** with: screenshot/GIF at the top, live demo link, architecture diagram, "how hallucination is prevented" section, local setup steps, `.env.example`.
 - 60-second demo video (Loom/OBS) — recruiters watch this, they don't clone repos.
 - A few meaningful tests (schema validation, cache hit logic) so the repo shows testing.
 - **DoD**: someone who has never met you understands the project in 90 seconds.
-

5. Cut list — build none of these

Collaborative editing · flight/hotel booking · payments · email notifications · multi-language · admin dashboard · mobile app · drag-and-drop reordering (nice, but day 14+) · chat interface with the planner.

Scope creep is the only real threat to this timeline.

6. Interview talking points to prepare

- Why retrieval-then-generate instead of pure generation.
 - How you enforce structured output and what happens when validation fails.
 - Your caching strategy and its measured effect on cost/latency.
 - Why you abstracted the LLM and Places providers.
 - What you'd do differently at 10,000 users (queue the generation, move to a job worker, per-tenant quotas, vector search over places).
-

7. Sources checked (September 2026)

- Google Maps Platform billing FAQ / pricing (March 2025 restructure, per-SKU free tiers)
- Google Maps Platform core services pricing list
- Gemini API free tier status and current model tiers
- Amadeus Self-Service test vs production environment docs
- Geoapify / Foursquare / Nominatim free tier documentation

API pricing moves fast — re-verify quotas on the official pages the day you sign up.